Nombres: Equipo 4 **Giovani Aldahir Patiño Rodriguez** **Aldo Rene Romero Parga** **Leonardo Alonso Muñiz Espinosa**	**Matrículas:** 2118917 2118748 2115635
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad: PIA (Proyecto Final)	**Actividad:** PIA. Implementación de una solución de software basada en lógica de negocio · VetCare 24
Fecha: 06/11/2026	
Repositorio: https://github.com/<USUARIO>/<REPO> · **API desplegada (Swagger UI):** https://<SERVICIO>.onrender.com/docs	

Bibliografía (Formato APA)

Facultad de Ingeniería Mecánica y Eléctrica. (2026). *Producto Integrador de Aprendizaje: Proyecto final — Implementación de una solución de software basada en lógica de negocio* [Documento del curso]. Universidad Autónoma de Nuevo León.

Equipo 4. (2026). *VetCare 24: Documento de requerimientos de negocio (BRD), historias de usuario con Gherkin, BPMN y glosario* [Documentos de las Unidades 2 y 3]. Facultad de Ingeniería Mecánica y Eléctrica, UANL.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions* [Entrada de blog]. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Okken, B. (2022). *Python testing with pytest: Simple, rapid, effective, and scalable* (2.ª ed.). Pragmatic Bookshelf.

Wiggins, A. (2017). *The twelve-factor app*. https://12factor.net/es/

Propósito de la Actividad

El propósito de este proyecto integrador es convertir el análisis y el diseño de VetCare 24 (BRD, historias de usuario con criterios Gherkin, BPMN, glosario y decisiones de arquitectura de las unidades 2 y 3) en una solución de software funcional, verificable y desplegable. La meta es que cada regla de negocio pueda rastrearse desde el documento de requerimientos hasta el código que la hace cumplir y la prueba automática que la comprueba, y no solo afirmar que «el programa corre».

Para lograrlo se construyó una API REST con Domain-Driven Design, Specification y State Machine; se escribieron pruebas unitarias y de integración con una cobertura muy superior al 80 % exigido; se contenerizó la solución con Docker Compose (base de datos, aplicación y pruebas); se separó la configuración del código mediante variables de entorno; y se automatizó la verificación con un pipeline de GitHub Actions y un despliegue con Swagger UI activo para la demostración.

Introducción

VetCare 24 es una clínica y hospital veterinario con atención 24/7 cuyo problema original eran los registros fragmentados: expedientes duplicados, consulta lenta de alergias en urgencias, vacunas sin recordatorios y una agenda que no distingue un procedimiento breve de una cirugía. En las unidades anteriores el trabajo se centró en entender el negocio, documentar sus reglas y diseñar el dominio. En este proyecto el reto cambia: hay que demostrar que el código respeta esas reglas, sobre todo en los casos límite, donde suelen esconderse los defectos más caros.

A partir de los artefactos del equipo se implementaron las 15 historias de usuario y las seis reglas de negocio RN-01 a RN-06 (inmutabilidad de la nota médica, prioridad del triage rojo, trazabilidad de biológicos, bloqueo por alergia, propiedad del historial y alta hospitalaria cruzada). Se resolvió además el paquete de datos del profesor (40 tutores, 20 servicios, 60 órdenes y 120 casos de prueba) y se validó con un oráculo independiente que no reutiliza el código del dominio. Al ejecutar las pruebas contra PostgreSQL real se detectó y corrigió un defecto de despliegue que SQLite no mostraba.

El reporte se organiza en el orden que pide el documento del proyecto: análisis del negocio (BRD, historias y Gherkin, BPMN, glosario y trazabilidad), arquitectura (DDD, patrones, SOLID, UML y ADRs), implementación y producción (Docker, variables de entorno, CI/CD y despliegue), paquete de datos, evidencias de pruebas, limitaciones y conclusiones. Al final se incluye un anexo que relaciona cada criterio de evaluación con su evidencia en el repositorio.

1. Resumen ejecutivo

VetCare 24 es una clínica y hospital veterinario con atención 24/7. A partir del BRD, las 15 historias de usuario con Gherkin, el BPMN y el diseño DDD elaborados en las unidades 2 y 3, se construyó una **API REST** en Python (FastAPI) con arquitectura limpia y **Domain Model** (ADR-001). Las seis reglas de negocio del BRD (RN-01…RN-06) viven en el dominio como objetos que se protegen solos; se implementaron los patrones **Specification** (AND/OR/NOT), **State Machine** y **Domain Events**, persistencia relacional (SQLite/PostgreSQL), contrato OpenAPI/Swagger, pruebas unitarias y de integración, contenedores Docker y CI/CD en GitHub Actions.

15 / 15	6 / 6	343	99.0 %	38
historias de usuario	reglas RN-01…06	pruebas automáticas	cobertura (dominio 100 %)	rutas OpenAPI

Además se resolvió el **paquete de datos** del caso (40 tutores, 20 servicios, 60 órdenes, 120 casos de prueba, 20 transiciones) y se validó con un **oráculo independiente** que no reutiliza el código del dominio.

2. Análisis del negocio (BRD)

Fuente: BRD del equipo (U2), refinado en `docs/BRD.md`; los originales en PDF están en `docs/analisis/originales/`.

2.1 Contexto y problema

Modelo operativo. VetCare 24 es una clínica y centro hospitalario veterinario con atención ininterrumpida (24/7): consultas generales y de especialidad, medicina preventiva, cirugía, laboratorio y hospitalización. Opera con cuenta familiar: un tutor puede tener varias mascotas y cada animal necesita un expediente individual, trazable e independiente.

Problema (AS-IS). Registros físicos y digitales fragmentados: expedientes duplicados; consulta lenta de antecedentes y alergias en urgencias; calendarios de vacunación sin recordatorios; agenda con duraciones estáticas y sin carril de emergencia.

Visión (TO-BE). Plataforma centralizada con expediente unificado tutor–mascotas, agenda por servicio, admisión inmediata de emergencias, notas inmutables con enmiendas auditadas y motor de recordatorios de vacunación.

2.2 Stakeholders y actores

Actor	Responsabilidad	Necesidad principal	Rol en la API
Recepción	Admisión, cobros, agenda, primer contacto	Buscar rápido sin duplicar fichas; agendar según la duración real	`RECEPCION`
Médico veterinario	Diagnóstico, prescripción, cirugía, altas	Historial al instante, notas SOAP inmutables, alertas de alergia	`VETERINARIO`
Personal clínico / enfermería	Cuidados en piso y monitoreo	Capturar signos vitales en rondas periódicas	`ENFERMERIA`
Propietario / tutor	Responsable legal y financiero	Recordatorios de refuerzos; indicaciones de tratamiento	Receptor de notificaciones
Director médico / administrador	Supervisión, auditoría, gestión operativa	Auditar enmiendas; monitorear ocupación y cumplimiento de citas	`DIRECTOR`

2.3 Alcance

Dentro del alcance	Fuera del alcance
Gestión de identidad (tutor, mascotas, microchip) · Agenda inteligente por servicio (15/30/45/90–120 min) · Admisión de urgencia · Expediente clínico con bloqueo tras firma y enmiendas · Seguridad farmacológica por alergias · Módulo profiláctico (lote obligatorio, refuerzos 21/28/365 d, avisos) · Hospitalización 24/7 con bitácora y alta cruzada · RBAC por perfiles	Facturación electrónica ante entidades tributarias · Nómina y asistencia · Tienda virtual · Consultas por videollamada

2.4 Reglas de negocio

ID	Regla (BRD)	Dónde se hace cumplir
RN-01	Inmutabilidad médica: la nota firmada se congela; solo se corrige con Nota de Enmienda (autor, fecha, motivo)	`Consulta._asegurar_editable`, `anexar_enmienda`
RN-02	Prioridad de triage rojo: salta lista de espera y trámites administrativos	`Consulta.admitir_urgencia`
RN-03	Trazabilidad de biológicos: lote y fabricante obligatorios antes de guardar una vacuna	`RegistroVacuna.aplicar`
RN-04	Bloqueo por alergia registrada al recetar el mismo principio activo (excepción con justificación bajo cédula, ADR-002)	`ServicioValidacionAlergias`
RN-05	El expediente pertenece al animal: la transferencia conserva el historial y reasigna contacto y cobro	`Paciente.transferir_a`
RN-06	Validación cruzada de alta: liberar la jaula exige alta médica firmada y cuenta/precuenta liquidada	`ServicioAltaHospitalaria`, `MAQUINA_HOSPITALIZACION`

2.5 Requerimientos funcionales y no funcionales

ID	Requerimiento	Implementación / verificación
RF-01	Búsqueda predictiva por teléfono, mascota, código o microchip	`GET /pacientes/buscar` · prueba US01/US12
RF-02	Hoja de triaje: peso, temperatura, motivo	`POST /pacientes/{id}/triaje`
RF-03	Citas con bloqueo automático por servicio	`ServicioAgenda`, `POST /citas`
RF-04	Cálculo automático del próximo refuerzo	`RegistroVacuna.aplicar`
RF-05	Recordatorio 48 h antes del refuerzo	`RefuerzoPorNotificarSpec`, `POST /recordatorios/procesar`
RF-06	Bitácora de constantes cada 2–4 h	`Hospitalizacion.registrar_constantes`
RNF-01	Expediente en menos de 3 s	prueba automatizada que mide el tiempo
RNF-02	Disponibilidad 24/7	`/health`, `HEALTHCHECK`, despliegue gestionado
RNF-03	Acceso por roles; recepción sin detalle clínico	`security.requiere`; pruebas 401/403
RNF-04	Vistas táctiles	No aplica a un backend REST (ver sección 12)

2.6 Criterios de éxito (KPIs)

Duplicados < 1 % · +40 % de retorno a refuerzos · ingreso de código rojo en < 15 s · 100 % de bitácoras con intervalos ≤ 4 h. El sistema provee los datos y las reglas (`GET /reportes/operativo`, `LecturaVencidaSpec`); la medición real requiere operación en producción.

3. Historias de usuario y criterios de aceptación (Gherkin)

Las 15 historias del equipo (5 módulos) con su escenario Gherkin, el componente que lo implementa y la prueba que lo verifica.

US	Historia	Criterio de aceptación (Gherkin)	Implementación y prueba

US01	Como Recepcionista, quiero registrar los datos del cliente y vincular múltiples mascotas a su ficha, para consolidar su historial bajo una misma cuenta familiar.	**Escenario:** Registro exitoso de mascota asociada
		Dado que el recepcionista ingresó los datos válidos del propietario
		Cuando completa nombre, especie, raza, peso y fecha de nacimiento de la mascota
		Entonces el sistema guarda la mascota vinculada al ID del cliente y genera su número de expediente único
			`PacienteService.registrar_mascota`
			`POST /pacientes`
			Prueba:
			`test_paciente_tutor.py::test_tutor_requiere_contacto_y_canal_valido`

US12	Como Recepcionista / Veterinario, quiero buscar pacientes por nombre, teléfono del dueño, microchip o código, para evitar la duplicación de fichas.	**Escenario:** Coincidencia predictiva de expedientes
		Dado que el recepcionista escribe el apellido del dueño y la raza en el buscador
		Cuando existen coincidencias previas registradas
		Entonces el sistema despliega una lista de coincidencias para verificar antes de crear un expediente nuevo
			`PacienteRepository.buscar`
			`GET /pacientes/buscar`
			Prueba:
			`test_api_clinica.py::test_us01_registro_y_busqueda_por_microchip_y_nombre`

US02	Como Recepcionista, quiero agendar citas bloqueando lapsos según el tipo de servicio, para evitar solapamientos y sobrecupos en las salas.	**Escenario:** Bloqueo de horario según servicio
		Dado que un cliente solicita una cita de «Cirugía Mayor»
		Cuando el recepcionista selecciona dicho servicio en el calendario
		Entonces el sistema reserva un bloque mínimo continuo de 90 minutos y asigna quirófano disponible
			`ServicioAgenda / Cita`
			`POST /citas`
			Prueba:
			`test_agenda.py::test_bloqueo_segun_servicio`

US03	Como Personal clínico, quiero admitir un paciente en estado crítico sin cita previa, para estabilizarlo de inmediato sin retrasos administrativos.	**Escenario:** Admisión de urgencia crítica
		Dado que arriba un paciente con traumatismo severo
		Cuando el usuario activa el botón «Admisión de Emergencia»
		Entonces el sistema notifica al veterinario en turno y abre un expediente temporal de intervención inmediata
			`Consulta.admitir_urgencia`
			`POST /urgencias`
			Prueba:
			`test_consulta.py::test_urgencia_no_exige_datos_administrativos_y_es_rojo`

US04	Como Asistente veterinario, quiero registrar el peso, temperatura y alergias reportadas al llegar, para actualizar el estado físico del animal previo a la auscultación.	**Escenario:** Detección de alerta por alergia
		Dado que el asistente ingresa «Penicilina» en el campo de alergias
		Cuando guarda los signos vitales de la visita
		Entonces el sistema muestra un indicador permanente de «Alerta de Alergia» en el encabezado del paciente
			`Paciente.registrar_triaje`
			`POST /pacientes/{id}/triaje`
			Prueba:
			`test_paciente_tutor.py::test_triaje_actualiza_peso_y_alergias`

US05	Como Médico Veterinario, quiero documentar el diagnóstico, anamnesis y tratamiento en una nota clínica, para certificar formalmente la atención brindada.	**Escenario:** Cierre de nota clínica inmutable
		Dado que el veterinario completó la nota de la consulta
		Cuando presiona «Firmar y Cerrar Nota»
		Entonces el registro se bloquea contra edición y se marca con firma, fecha y cédula del profesional
			`Consulta.firmar_y_cerrar`
			`POST …/nota/firmar`
			Prueba:
			`test_consulta.py::test_firmar_congela_y_registra_cedula`

US06	Como Médico Veterinario, quiero recetar fármacos dosificados según peso del paciente, para emitir una receta clara y libre de riesgos toxicológicos.	**Escenario:** Bloqueo por prescripción de sustancia alérgena
		Dado que la mascota tiene registrada alergia activa a un principio activo
		Cuando el médico intenta agregar dicho fármaco a la receta digital
		Entonces el sistema interrumpe la acción con una advertencia crítica de contraindicación
			`ServicioValidacionAlergias`
			`POST …/recetas`
			Prueba:
			`test_consulta.py::test_rn04_bloquea_alergeno_y_no_modifica_receta`

US07	Como Sistema, quiero calcular la fecha del próximo refuerzo al aplicar una vacuna, para mantener actualizado el calendario preventivo.	**Escenario:** Creación de cita profiláctica sugerida
		Dado que el veterinario registra la aplicación de la vacuna «Quíntuple Canina»
		Cuando se valida el lote y la dosis
		Entonces el sistema agenda un recordatorio de refuerzo exactamente 21 días después
			`RegistroVacuna.aplicar`
			`POST /vacunas`
			Prueba:
			`test_profilaxis.py::test_refuerzo_exacto`

US08	Como Propietario, quiero recibir avisos por WhatsApp/correo previo a las vacunas de mi mascota, para cumplir en tiempo con su cuadro preventivo.	**Escenario:** Despacho automatizado de notificación
		Dado que faltan 48 horas para la fecha teórica de un refuerzo
		Cuando el proceso programado evalúa las fechas pendientes
		Entonces se despacha un mensaje al canal preferido del dueño
			`RefuerzoPorNotificarSpec`
			`POST /recordatorios/procesar`
			Prueba:
			`test_api_clinica.py::test_us08_recordatorio_48h_antes_se_envia_una_sola_vez`

US09	Como Médico Veterinario, quiero transferir un paciente a hospitalización indicando cama y protocolo, para asegurar monitoreo continuo.	**Escenario:** Asignación de jaula/box hospitalario
		Dado que se dictaminó orden de internamiento
		Cuando el médico selecciona una jaula desocupada y carga el plan de fluidoterapia
		Entonces el estado del paciente cambia a «Hospitalizado» y se activa la hoja de enfermería 24/7
			`ServicioIngresoHospitalario`
			`POST /hospitalizaciones`
			Prueba:
			`test_hospitalizacion.py::test_ingreso_asigna_jaula_compatible`

US10	Como Enfermero veterinario, quiero volcar las lecturas periódicas de constantes vitales en la bitácora, para monitorear tendencias y evolución.	**Escenario:** Registro de signos fuera de rango
		Dado que el paciente hospitalizado se encuentra bajo monitoreo periódico
		Cuando el enfermero captura una frecuencia cardíaca crítica anormal
		Entonces el sistema emite una alerta de signos fuera de rango
			`Hospitalizacion.registrar_constantes`
			`POST …/constantes`
			Prueba:
			`test_hospitalizacion.py::test_ventana_4h_estable_y_2h_critico`

US11	Como Médico Veterinario, quiero emitir la orden de alta con indicaciones de cuidado en casa, para permitir que el dueño proceda al retiro del animal.	**Escenario:** Validación de alta integral
		Dado que el médico cargó la nota de egreso y la firma de alta
		Cuando recepción valida que los insumos y días de hospitalización fueron auditados
		Entonces el estado pasa a «Alta Médica / egreso concluido» y se libera la jaula
			`ServicioAltaHospitalaria`
			`POST …/alta-medica · …/liquidacion`
			Prueba:
			`test_hospitalizacion.py::test_alta_requiere_ambas_confirmaciones_sin_importar_orden`

US13	Como Director Médico, quiero añadir notas aclaratorias sobre registros antiguos sellados, para enmendar omisiones sin destruir el historial original.	**Escenario:** Anexo de nota de aclaración
		Dado un expediente con una consulta cerrada
		Cuando el médico agrega una «Nota de Enmienda» justificada
		Entonces el sistema vincula la aclaración manteniendo intacta la versión previa
			`Consulta.anexar_enmienda`
			`POST …/enmiendas`
			Prueba:
			`test_consulta.py::test_enmienda_solo_sobre_nota_firmada_y_conserva_original`

US14	Como Médico Veterinario, quiero solicitar perfiles hematológicos y anexar los resultados al expediente, para correlacionar estudios diagnósticos.	**Escenario:** Adjuntar reporte de laboratorio **Dado** que el laboratorio externo remitió el archivo de un hemograma **Cuando** el veterinario sube el archivo al expediente **Entonces** el sistema asocia el documento a la consulta y alerta al médico tratante	`Consulta.adjuntar_resultado` `POST …/laboratorio` Prueba: `test_consulta.py::test_us14_solicitar_y_adjuntar_resultado_emite_alerta_al_medico`
US15	Como Administrador de la clínica, quiero consultar métricas de ocupación de jaulas y vacunas aplicadas semanalmente, para planificar compras y personal.	**Escenario:** Extracción de KPIs semanales **Dado** que el administrador selecciona el rango de los últimos 7 días **Cuando** solicita la generación del reporte **Entonces** la plataforma entrega la tasa de ocupación, citas canceladas y vacunas aplicadas	`ReporteService.operativo` `GET /reportes/operativo` Prueba: `test_api_clinica.py::test_reporte_operativo_solo_director`

4. Proceso de negocio (BPMN)

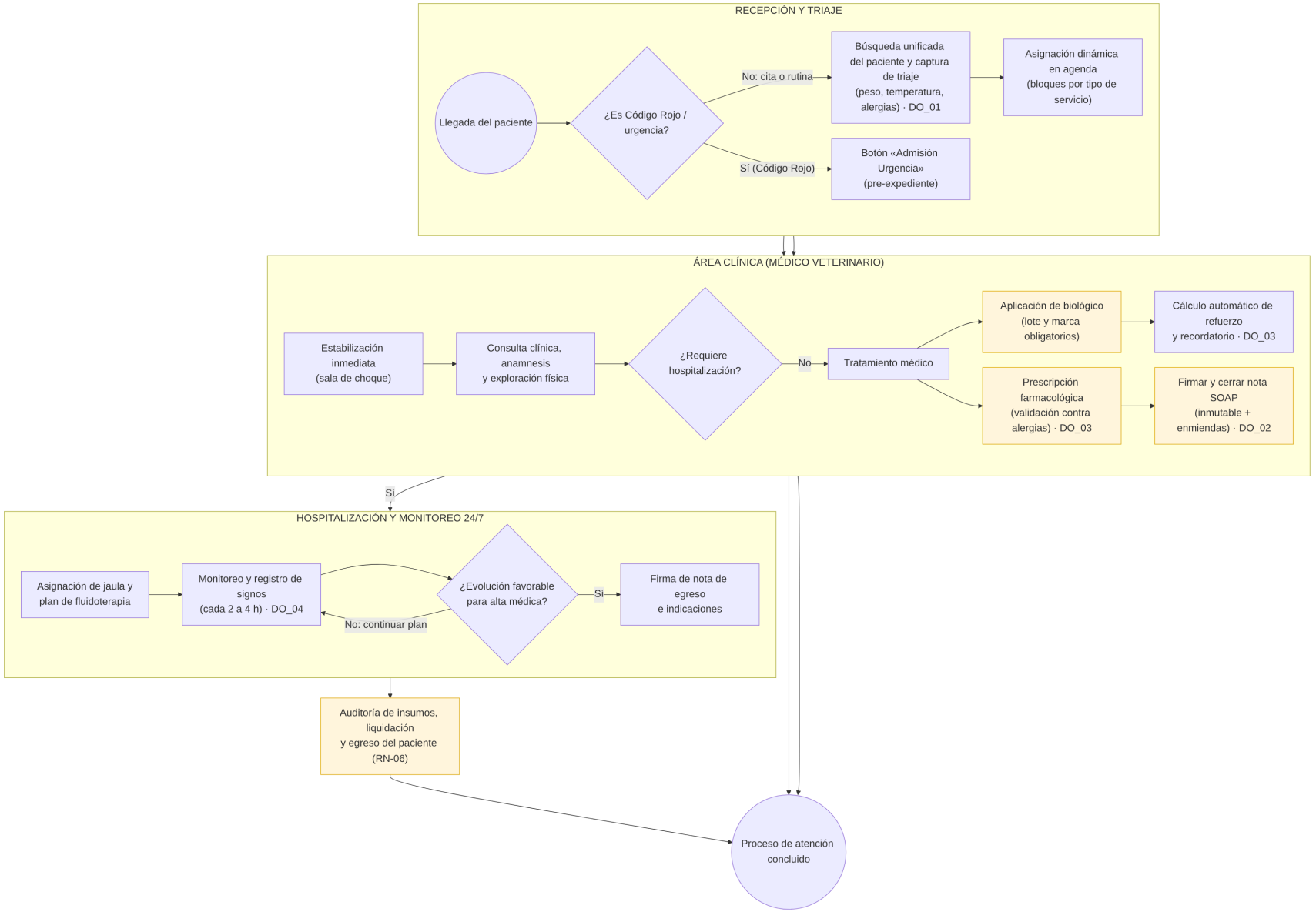


Figura 1. BPMN del proceso central de VetCare 24 (redibujado a partir del BPMN del equipo)

Proceso central de atención, internamiento y profilaxis, en tres carriles. Los objetos de datos DO_01…DO_04 corresponden a la ficha de admisión, el expediente electrónico, la receta con avisos y la bitácora hospitalaria. Las actividades sombreadas son las que materializan reglas de negocio.

Actividad BPMN	Regla / historia	Componente
Botón «Admisión Urgencia» (Código Rojo)	RN-02 · US03	`Consulta.admitir_urgencia` → `POST /urgencias`
Búsqueda unificada + triaje (DO_01)	US12 · US04	`PacienteRepository.buscar`, `Paciente.registrar_triaje`
Asignación dinámica en agenda	US02 · RF-03	`ServicioAgenda`, `SalaLibreSpec`
Prescripción con validación contra alergias (DO_03)	RN-04 · US06	`ServicioValidacionAlergias`
Firmar y cerrar nota SOAP (DO_02)	RN-01 · US05 · US13	`Consulta.firmar_y_cerrar`, `NotaEnmienda`
Aplicación de biológico → refuerzo y recordatorio	RN-03 · US07 · US08	`RegistroVacuna.aplicar`, `RefuerzoPorNotificarSpec`
Asignación de jaula y fluidoterapia	US09	`ServicioIngresoHospitalario`
Monitoreo cada 2–4 h (DO_04)	RF-06 · US10	`Hospitalizacion.registrar_constantes`, `LecturaVencidaSpec`
Nota de egreso, auditoría de insumos y liquidación	RN-06 · US11	`ServicioAltaHospitalaria`, `MAQUINA_HOSPITALIZACION`

5. Glosario de dominio

Los 20 términos del glosario del equipo y el elemento del modelo donde viven (lenguaje ubicuo: el código usa los mismos nombres).

#	Término	Definición	En el modelo
1	**Propietario (Tutor)**	Persona legalmente responsable de la mascota y de las obligaciones financieras de la atención.	`Tutor` (`tutor.py`)
2	**Paciente (Mascota)**	Animal registrado que recibe servicios preventivos, terapéuticos o quirúrgicos.	`Paciente`, `TutorResponsable`
3	**Expediente Clínico Digital**	Conjunto cronológico de notas, diagnósticos, resultados y tratamientos de un paciente.	`Consulta` + `GET /pacientes/{id}/expediente`
4	**Triage Veterinario**	Clasificación rápida por gravedad (Rojo = emergencia, Amarillo = urgencia, Verde = rutina).	`NivelTriage`, `HojaTriaje`

5	**Anamnesis**	Información retrospectiva que el veterinario obtiene del propietario sobre el motivo de consulta.	`NotaSOAP.subjetivo`
6	**Signos Vitales**	Variables fisiológicas objetivas: temperatura, frecuencia cardíaca y respiratoria, peso.	`ConstantesVitales`, `Peso`
7	**Nota de Evolución**	Asiento médico obligatorio con método SOAP en cada encuentro clínico.	`NotaSOAP`, `EstadoNota`
8	**Nota de Enmienda**	Addendum oficial que corrige o detalla un registro cerrado sin vulnerar su inmutabilidad.	`NotaEnmienda` (RN-01)
9	**Calendario Profiláctico**	Cronograma de inmunizaciones y desparasitaciones.	`RegistroVacuna`, `ProgramacionRefuerzo`
10	**Refuerzo (Booster)**	Dosis subsecuente de un biológico tras un intervalo específico.	`IntervaloRefuerzo`, `EsquemaRefuerzo` (21/28/365 d)
11	**Lote de Biológico**	Código del fabricante registrado para trazabilidad y farmacovigilancia.	`LoteBiologico`, `MarcaVacuna` (RN-03)
12	**Prescripción Terapéutica**	Instrucción formal con fármaco, dosis por kg, vía, frecuencia y duración.	`ItemReceta`, `MedicamentoDosificado`
13	**Alerta de Alergia**	Mecanismo que bloquea o advierte la administración de sustancias incompatibles.	`Alergeno`, `ServicioValidacionAlergias` (RN-04)
14	**Hospitalización**	Internamiento para observación continua, fluidoterapia o terapia intensiva.	`Hospitalizacion`, `Jaula`
15	**Bitácora de Monitoreo**	Registro periódico (2–4 h) de signos vitales, ingesta y medicación.	`LecturaBitacora`, `LecturaVencidaSpec`
16	**Alta Médica**	Dictamen del veterinario tratante que certifica la superación del criterio de internamiento.	`ServicioAltaHospitalaria`, `AltaMedicaAutorizadaEvent` (RN-06)
17	**Cita Diferenciada**	Reserva cuya duración y recursos varían según el procedimiento.	`Cita`, `ServicioAgenda`
18	**Consentimiento Informado**	Documento firmado por el propietario que autoriza intervenciones o riesgos.	Fuera de alcance (ver sección 12)
19	**Registro Duplicado**	Una misma mascota o tutor con más de un identificador, dispersando su historial.	`MicrochipCode` único + `PacienteRepository.buscar`
20	**KPI Clínico**	Métrica de gestión: ocupación de jaulas, retención de vacunas, tiempos de espera.	`ReporteService.operativo` (US15)

6. Trazabilidad

Cada regla, historia, escenario Gherkin y elemento BPMN se enlaza con su clase de dominio, su código y sus pruebas en `docs/TRAZABILIDAD.md` (24 filas), generado desde una tabla de datos. La prueba `test_trazabilidad.py` falla si algún archivo o símbolo citado ya no existe o si el documento está desactualizado, de modo que la matriz no puede desalinearse del código.

Cadena	Ejemplo (RN-04)
BRD → User Story → Gherkin	RN-04 → US06 → «Bloqueo por prescripción de sustancia alérgena»
BPMN	Prescripción farmacológica (validación activa contra alergias, DO_03)
Dominio	`ServicioValidacionAlergias`, `AlergenoCoincideSpec`, `BloqueoAlergiaDisparadoEvent`
Código	`services.py::ConsultaService` · `POST /consultas/{id}/recetas` (409)
Pruebas	unitarias del servicio de alergias + integración del endpoint (409 / 201 con justificación)

7. Arquitectura

7.1 Decisión: Domain Model (ADR-001)

Se comparó Transaction Script, Table Module y Domain Model. Se eligió **Domain Model (DDD)** porque las reglas RN-01…RN-06 son invariantes entrelazadas con ciclo de vida de estados: en un modelo rico el agregado las hace cumplir y se prueban sin base de datos ni servidor web.

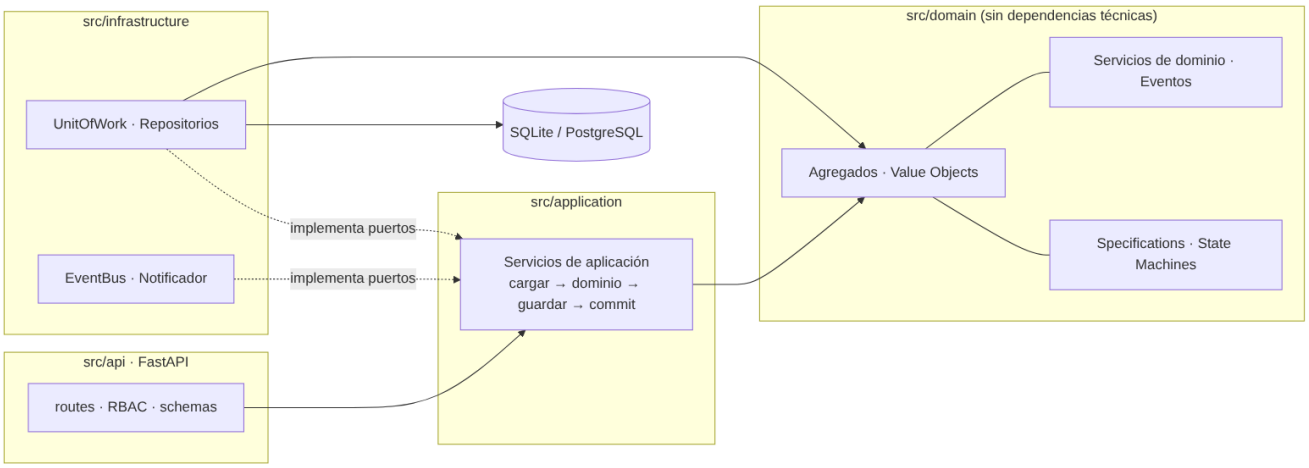


Figura 2. Capas y regla de dependencia (api → application → domain ← infrastructure)

7.2 Agregados y patrones

Agregado (raíz)	Invariantes
Paciente	microchip único; alergias sin duplicar; transferencia conserva historial
Consulta (SOAP, enmiendas, receta, estudios)	nota firmada inmutable; urgencia sin trámites; resultados de laboratorio se anexan sin editar la nota
Hospitalizacion + Jaula	lecturas cada 4 h (2 h en crítico); alta cruzada
RegistroVacuna	lote y marca obligatorios; refuerzo 21/28/365 días
Cita	duración por servicio (15/30/45/90–120 min), sala sin solapes, cancelación tardía < 4 h
OrdenServicio (paquete de datos)	`puede_procesarse`, beneficio de referencia, máquina de estados

Los agregados se referencian solo por identificador (`paciente_id`, `tutor_id`), de modo que cada transacción modifica un único agregado.

- **Specification** con `&`, `|`, `~` y `reasons()`; p. ej. `puede_procesarse = TutorActivo & OrdenConLineas & OrdenAutorizada` y `beneficio = Preferente & TotalMínimo & ~Promoción`.
- **State Machine** por tabla (estado × acción → estado) para órdenes, hospitalización y citas; una transición inválida produce `InvalidStateTransition` (HTTP 409).
- **Domain Events** publicados *después* del commit (ADR-006); suscriptor activo: alerta al médico al recibir un resultado de laboratorio.
- **Repository + Unit of Work**; **Service Layer** delgada (solo orquesta).



Figura 3. Máquina de estados de Hospitalizacion (RN-06)

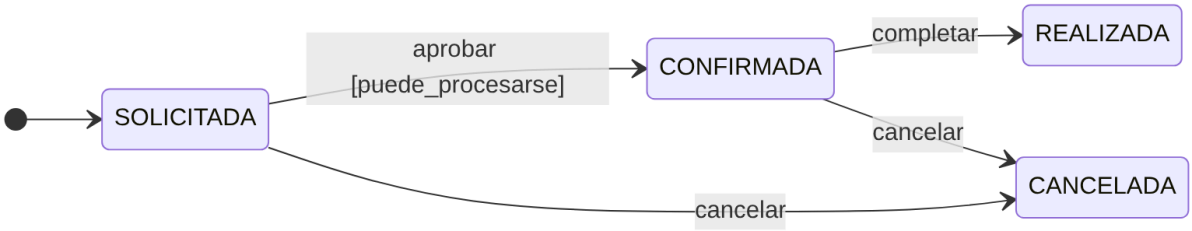


Figura 4. Máquina de estados de OrdenServicio (paquete de datos)

7.3 Patrones de diseño: selección y justificación

Patrón	Problema del negocio que resuelve	Dónde está	Alternativa descartada
Domain Model (DDD)	Seis reglas entrelazadas con ciclo de vida (RN-01…06) que no deben filtrarse a BD ni UI.	`src/domain`: agregados con sus Value Objects	Transaction Script y Table Module (ADR-001)

Specification (AND/OR/NOT)	Reglas compuestas del BRD y del paquete de datos que deben explicar *por qué* fallan.	`specifications.py`, `agenda.py`; `puede_procesarse = TutorActivo & OrdenConLineas & OrdenAutorizada`	`if` anidados en servicios (ADR-005)
State Machine	Orden, hospitalización y cita tienen estados con transiciones válidas e inválidas (RN-06).	`StateMachine` en `common.py`; tablas en cada agregado	Banderas booleanas sueltas; librería externa (ADR-005)
Domain Events	Reaccionar a hechos (resultado de laboratorio, bloqueo por alergia) sin acoplar módulos.	`events.py` (23 eventos); `InMemoryEventBus`	Llamadas directas; broker externo (ADR-006)
Repository + Unit of Work	Persistir agregados completos y atómicos sin que el dominio conozca SQLAlchemy.	`repositories.py`, `uow.py`	ORM con tablas hijas (ADR-003)
Service Layer	Orquestar casos de uso (transacción, eventos) sin reglas de negocio.	`src/application/services.py`	Lógica en los controladores
Domain Service	Reglas que involucran varios agregados (alergia vs receta; alta cruzada).	`servicios.py`	Métodos en un agregado que conocería a otro

7.4 Diagramas de clases del dominio (UML)

El modelo se presenta por agregado (rombo lleno = composición dentro del agregado; flecha punteada = referencia por identificador a otro agregado). Todas las raíces heredan de `AggregateRoot`, que registra los eventos de dominio.

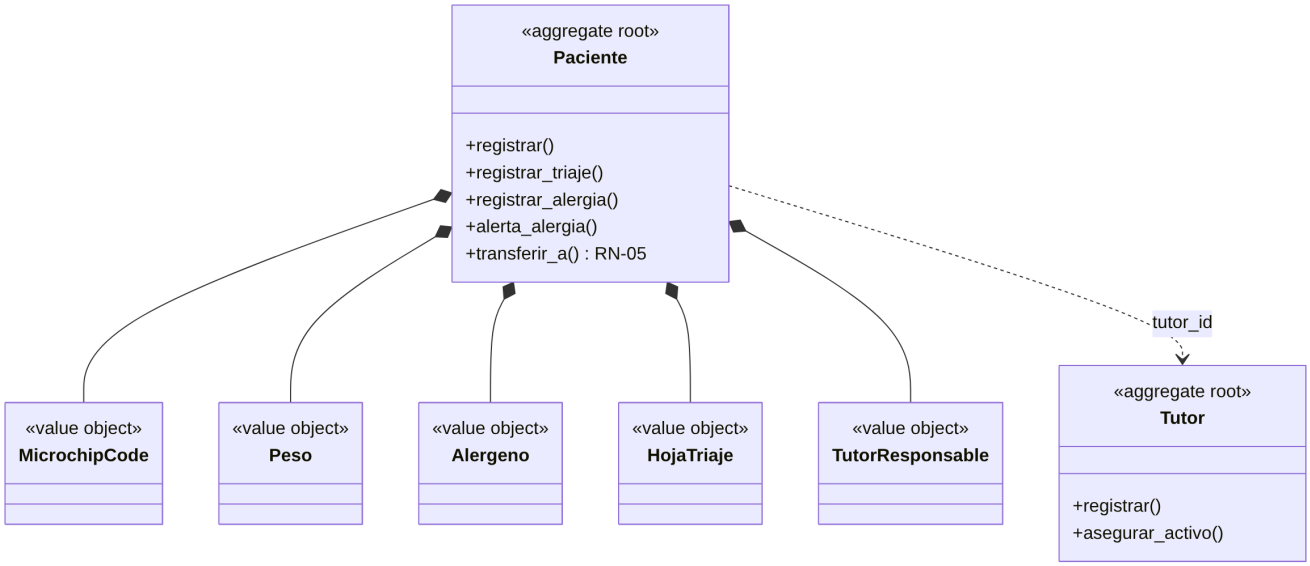


Figura 5. Tutor y Paciente con sus Value Objects (RN-05)

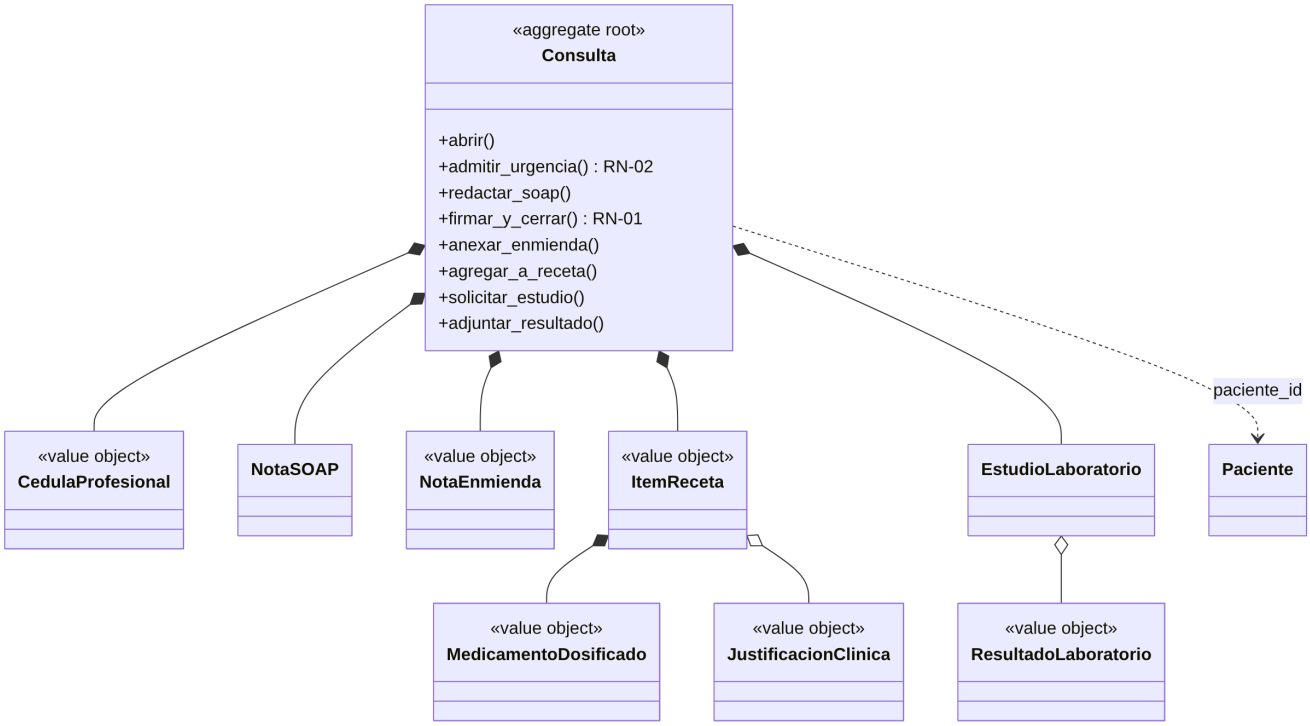


Figura 6. Consulta: nota SOAP, receta, enmiendas y laboratorio (RN-01, RN-02, RN-04)

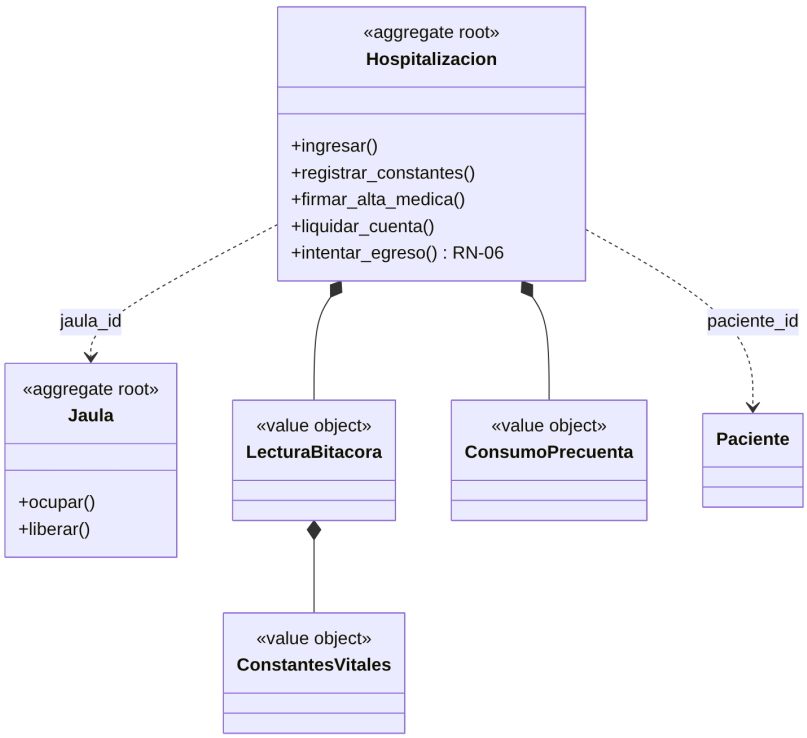


Figura 7. Hospitalización y Jaula (RN-06)

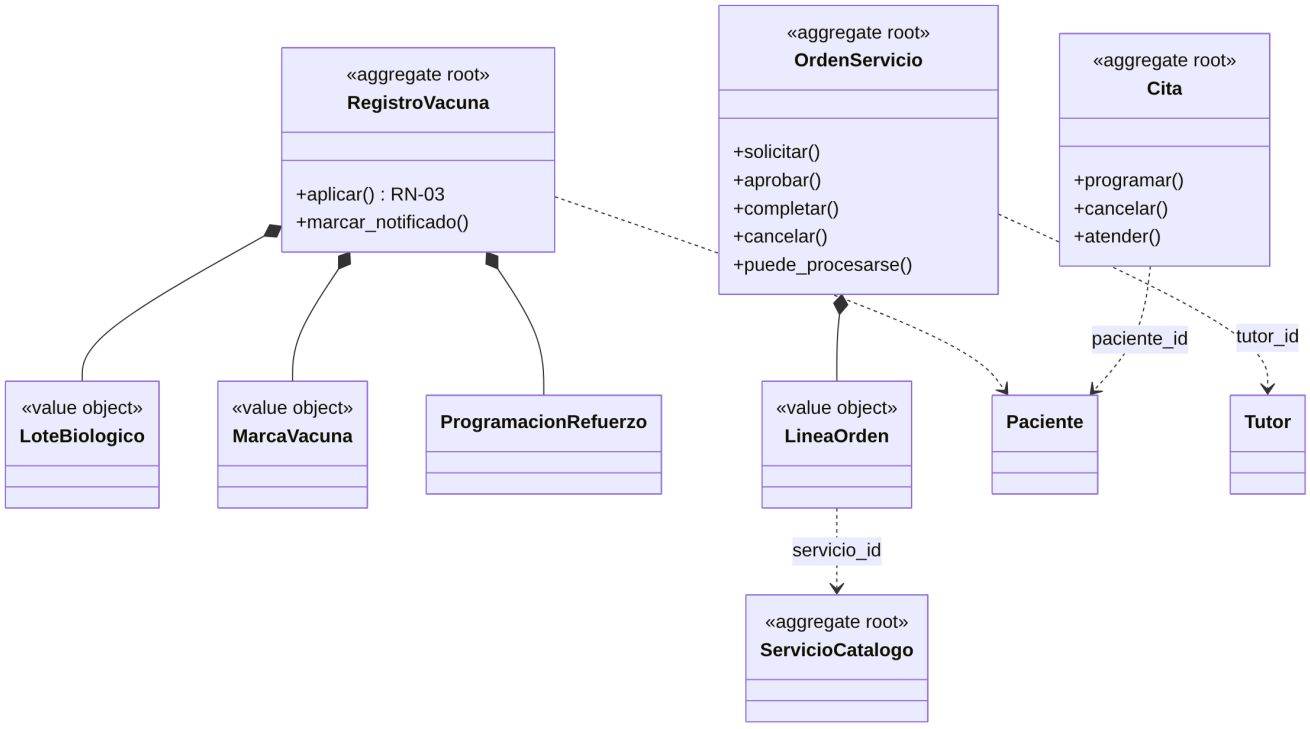


Figura 8. Vacunas (RN-03), Citas y Órdenes de servicio

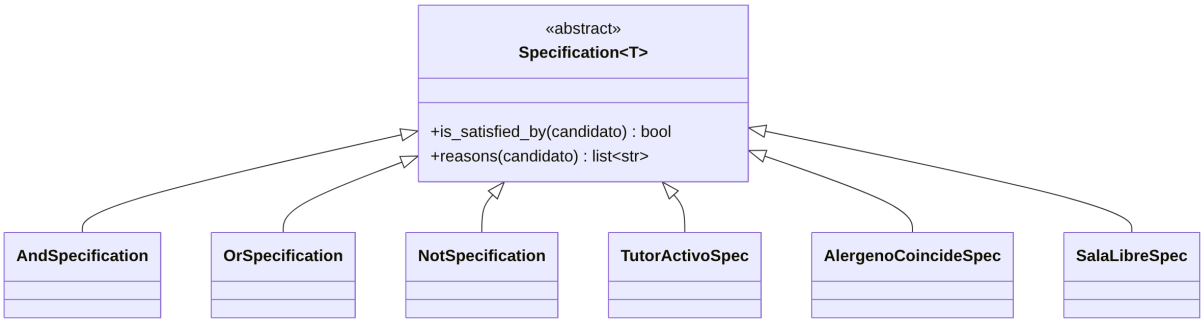


Figura 9. Jerarquía de Specification

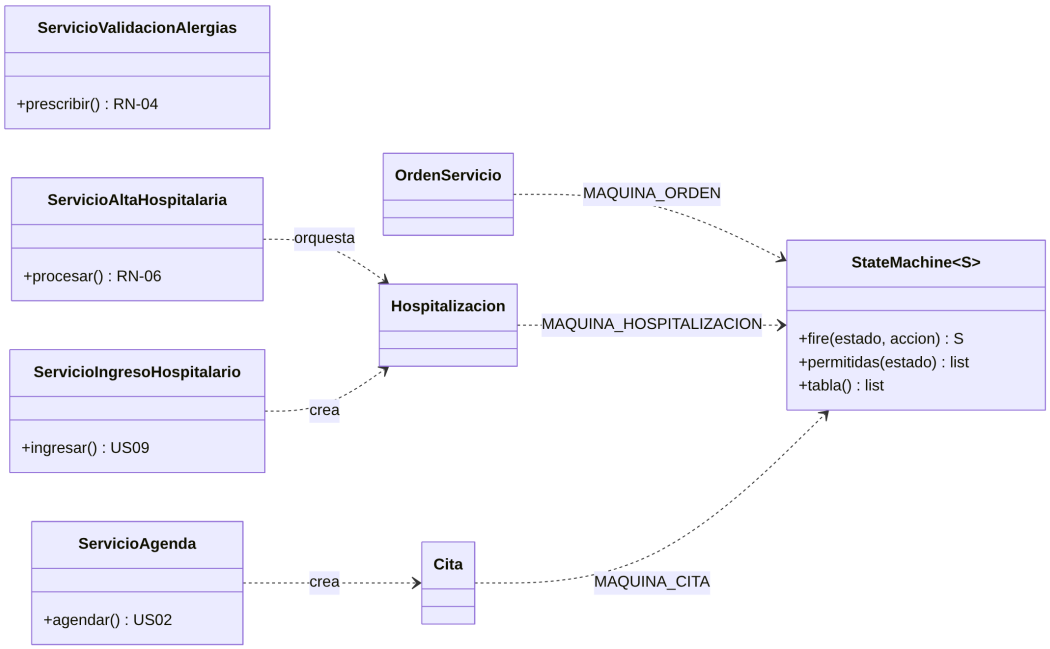


Figura 10. State Machine y servicios de dominio

7.5 Principios SOLID

Principio	Aplicación en el código	Verificación
S Responsabilidad única	Un agregado por conjunto de invariantes; un servicio de dominio por regla (`ServicioValidacionAlergias`, `ServicioIngresoHospitalario`, `ServicioAltaHospitalaria`); servicios de aplicación solo orquestan; controladores sin lógica	pruebas unitarias por pieza
O Abierto/cerrado	Una regla nueva es una `Specification` nueva compuesta con `&`, `|`, `~`; un estado nuevo es una fila de la tabla de `StateMachine`; un agregado nuevo es una subclase de `DocRepository`	`test_specification`, `test_state_machine`
L Sustitución de Liskov	`And/Or/NotSpecification` sustituyen a `Specification`; cualquier `Notificador` o `EventBus` cumple el puerto; SQLite y PostgreSQL son intercambiables	suite completa en ambos motores; `test_arquitectura`

I Segregación de interfaces	Puertos pequeños y separados: `EventBus`, `Notificador`, `UnitOfWork`	`src/application/ports.py`
D Inversión de dependencias	`application` depende de puertos; `infrastructure` los implementa; `api/container.py` es la única raíz de composición	`test_arquitectura` analiza los `import` de todo `src/` y falla si una capa depende hacia afuera

Detalle con fragmentos de código en `docs/architecture/SOLID.md`. Límites reconocidos: `UnitOfWork` expone sus repositorios sin tipar uno a uno, y algunos servicios de aplicación son grandes.

7.6 Registros de decisión (ADR)

Cada ADR documenta contexto, decisión, alternativas consideradas y consecuencias.

ADR	Decisión
001	Domain Model / DDD con 4 agregados, servicios de dominio y Service Layer delgada (del equipo, U3)
002	Bloqueo por alergia con excepción justificada bajo cédula (RN-04)
003	Persistencia: un agregado = una fila (columnas indexadas + documento JSON/JSONB)
004	RBAC demostrativo por encabezado `X-Rol` (RNF-03)
005	Specification y State Machine explícitas
006	Eventos de dominio publicados solo tras el commit

8. Implementación

Aspecto	Detalle
Stack	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, pytest + pytest-cov, PostgreSQL 16 / SQLite
Estructura	`src/domain` · `src/application` · `src/infrastructure` · `src/api` · `tests` · `docs`
Persistencia	Una tabla por agregado: columnas para consultar + columna `doc` (JSONB en PostgreSQL) serializada por un codec propio; el dominio no conoce el ORM
API	38 rutas (39 operaciones) en Swagger UI en español: cada recurso explica qué hace, su regla y los roles que pueden usarlo; los cuerpos traen ejemplos válidos y el rol DIRECTOR entra precargado; errores 400 (dato inválido) · 404 · 409 (regla de negocio / estado) · 422 (contrato); OpenAPI en `/openapi.json`, Swagger UI en `/docs`
Seguridad	RBAC por rol (RECEPCION, VETERINARIO, ENFERMERIA, DIRECTOR) con encabezado `X-Rol` — **demostrativo, no es autenticación**
Configuración	12-Factor: `DATABASE_URL`, `SEED_ON_START`, `DATA_DIR`, `ENVIRONMENT`, `PORT`; `.env.example` sin secretos
Contenedores	`Dockerfile` multi-etapa (base / test / app, usuario sin privilegios, `HEALTHCHECK`); `docker-compose.yml` con servicios *db* (PostgreSQL), *app* y *tests*
CI/CD	GitHub Actions: pruebas contra PostgreSQL de servicio, cobertura ≥ 80 %, exportación de OpenAPI, build de imagen, pruebas en Compose y *deploy hook* a Render

Despliegue	`render.yaml` (Blueprint: PostgreSQL + servicio web Docker); Swagger UI en `/docs`

9. Producción: Docker Compose, 12-Factor, CI/CD y despliegue

Requisito del PIA	Cómo se cumple	Evidencia
Docker Compose para DB + App + Tests	`docker-compose.yml`: servicio `db` (PostgreSQL 16 con healthcheck), `app` (API en :8000, espera a la BD sana) y `tests` (pytest con cobertura; perfil `tests-pg` contra PostgreSQL). Imagen multi-etapa (base / test / app) con usuario sin privilegios y `HEALTHCHECK`	`docker compose up --build` · `docker compose run --rm tests`
Variables de entorno (12-Factor)	Toda la configuración sale del entorno: `DATABASE_URL`, `SEED_ON_START`, `DATA_DIR`, `ENVIRONMENT`, `PORT`. `.env.example` sin secretos; `.env` excluido por `.gitignore` y `.dockerignore`	`.env.example`, `src/infrastructure/config.py`
Pipeline CI/CD activo	GitHub Actions (`.github/workflows/ci.yml`): pytest contra un PostgreSQL de servicio con umbral de cobertura ≥ 80 %, exportación del OpenAPI, build de la imagen, pruebas dentro de Compose y deploy hook de Render	pestaña *Actions* del repositorio
Despliegue funcional	`render.yaml` (Blueprint): base PostgreSQL + servicio web Docker con `SEED_ON_START=true` y `/health` como health check	URL pública (portada y README)
Swagger UI / OpenAPI activo	Swagger UI en `/docs` (en español, rol precargado y formulario por campos) y contrato en `/openapi.json`, exportado a `docs/api/openapi.json` en cada CI	Figuras 11 y 12



VetCare 24 API

/openapi.json

API REST de **VetCare 24**, clínica y hospital veterinario 24/7 (PIA de Componentes de Lógica de Negocios, FIME-UANL, Equipo 4). Cada recurso aplica una regla del negocio definida en el BRD, las User Stories y el BPMN (**RN-01 … RN-06**).

Cómo usar cada operación (3 pasos)

1. **Abre** una operación haciendo clic en su barra de color (por ejemplo `POST /tutores`).
2. **Llena el formulario**: un cuadro por dato (nombre, teléfono, etc.), todos marcados en **azul** y ya con un ejemplo válido. Si la operación pide un `id` en *Parámetros*, pégalo ahí: es el `id` que devolvió el paso anterior. Luego pulsa **▶ Ejecutar**.
3. **Lee el resultado** en *Respuesta del servidor*, justo debajo: código **2xx** = funcionó; **4xx** = algo falló y el mensaje dice qué. Lo que aparece más abajo en gris (*Ejemplo*) es solo documentación, no tu resultado.

El rol ya viene elegido (`DIRECTOR` , puede usar todo). Para cambiarlo pulsa **Autorizar** (arriba a la derecha).

Recorrido sugerido (cada paso usa el `id` que devolvió el anterior)

#	Recurso	Qué demuestra
1	`POST /tutores` → `POST /pacientes`	Alta de tutor y mascota (US01)
2	`POST /pacientes/{id}/alergias`	Registrar alergia a *penicilina*
3	`POST /consultas`	Abrir consulta con cédula del veterinario
4	`POST /consultas/{id}/recetas`	**409 RN-04**: bloqueo por alergia
5	misma receta + `justificacion` y `cedula_justificante`	**201**: excepción clínica (ADR-002)
6	`PUT …/nota` → `POST …/nota/firmar`	Redactar y firmar la nota SOAP (US05)
7	`PUT …/nota` otra vez	**409 RN-01**: una nota firmada no se edita
8	`POST …/enmiendas`	**201**: la corrección va como nota de enmienda (US13)
9	`POST /hospitalizaciones` (jaula `JP-01`)	Ingreso a hospitalización (US09)
10	`POST …/alta-medica` → `POST …/liquidacion`	**RN-06**: la jaula se libera solo con alta firmada **y** cuenta liquidada
11	`GET /ordenes/VET-0001` → `POST …/transiciones`	Datos del profesor, Specification y State Machine

Figura 11. Swagger UI en `/docs`: guía de uso y recorrido sugerido (captura local; sustituir por la del despliegue)

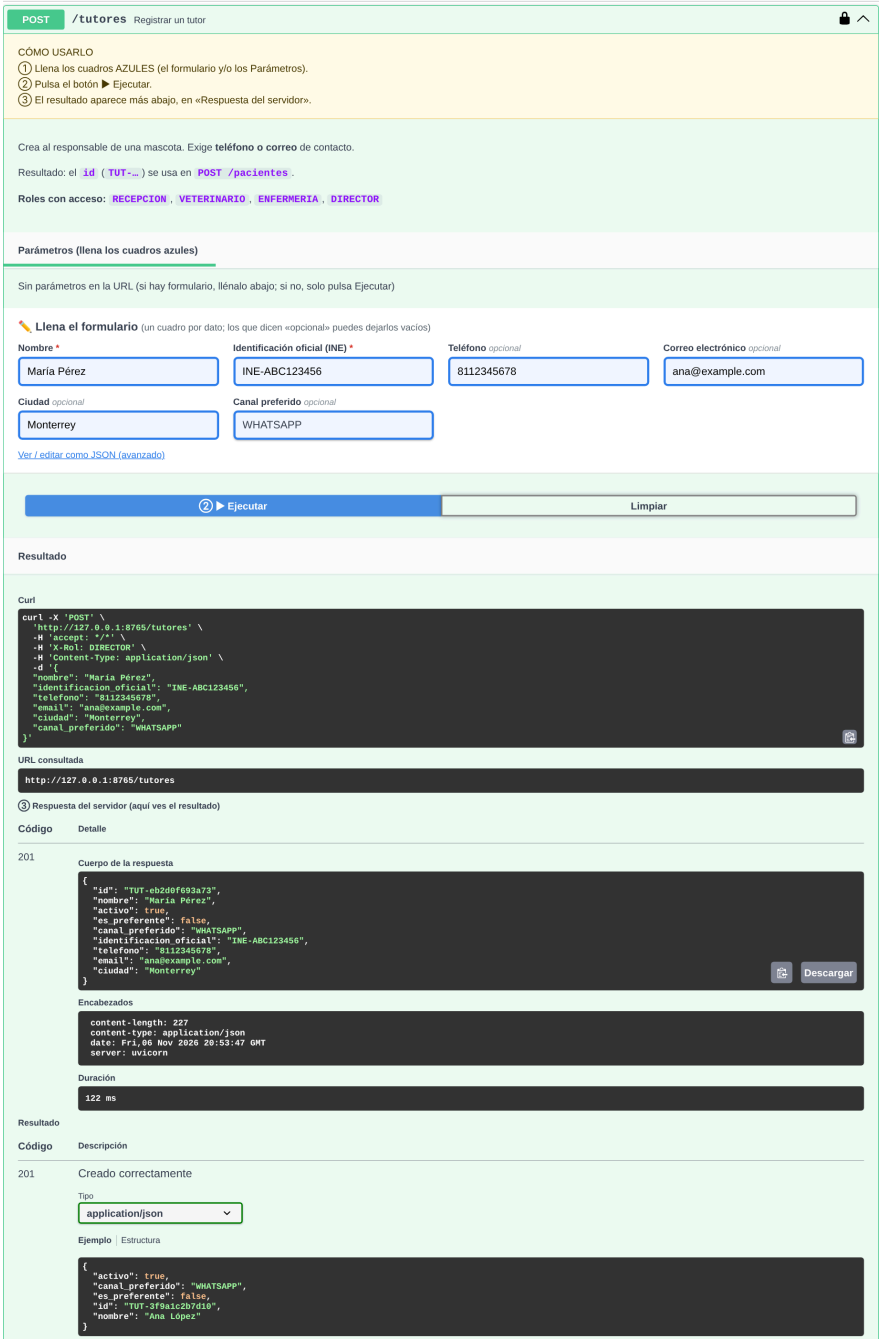


Figura 12. Una operación con formulario por campos, ejecutada con éxito (201): los datos viajan en la petición real

10. Paquete de datos VetCare 24

Los CSV (40 tutores, 20 servicios, 60 órdenes con 117 líneas de detalle, 120 casos, 20 transiciones) se cargan con `SEED_ON_START=true` y se ejecutan contra el dominio real con `scripts/ejecutar_casos_prueba.py`.

Regla evaluada	Resultado sobre las 60 órdenes

`validar_operacion` (tutor activo AND ≥ 1 línea AND autorizada)	43 válidas · 17 inválidas
`evaluar_beneficio` (preferente AND total ≥ 1 500 AND NOT promoción)	9 aplican · 51 no aplican
Transiciones (20)	aprobar/cancelar válidos; `completar` y `saltar_estado` desde SOLICITADA → 409; `repetir_aprobacion` → 409 en la 2.ª aprobación

Oráculo independiente. `test_oraculo_paquete_datos.py` recalcula cada total y cada veredicto únicamente con `csv` y `Decimal`, sin importar `src.domain`, y los compara contra las respuestas de la API: 120 casos + 20 transiciones. Así el código no se valida contra sí mismo.

> El `LEEME.txt` del paquete indica que sus reglas son ilustrativas y que las diferencias con el BRD deben documentarse. Ver `docs/DIFERENCIAS_PAQUETE_DATOS.md` (p. ej. interpretación de estados terminales, seis órdenes históricas con un servicio deshabilitado, dos órdenes sin líneas).

11. Evidencia de pruebas

Medición	Resultado
Pruebas automáticas	**343 pasan** en SQLite en memoria y en PostgreSQL 16 real (`TEST_DATABASE_URL`)
Cobertura total	**99.0 %** (requerido ≥ 80 %)
Por capa	dominio **100 %** · API 99.0 % · infraestructura 99.0 % · aplicación 96.3 %
Tipos	unitarias de dominio sin BD · integración vía API REST · oráculo del paquete de datos · trazabilidad verificada

Hallazgo relevante: al ejecutar la suite contra PostgreSQL real se detectó que SQLAlchemy 2 interpreta `postgresql://` como el driver psycopg v3 mientras el proyecto usa psycopg2; habría fallado el despliegue. Se corrigió con `normalizar_url` (que además convierte `postgres://` de Render) y quedó cubierto por pruebas.

Cobertura por módulo (`pytest --cov=src`)

```
src/__init__.py                      0      0   100%
src/api/__init__.py                  0      0   100%
src/api/container.py                38      0   100%
src/api/docs.py                     58      3    95%
src/api/main.py                     33      0   100%
src/api/routes.py                  129      2    98%
src/api/schemas.py                 225      1    99%
src/api/security.py                 19      0   100%
src/application/__init__.py          0      0   100%
src/application/ports.py            17      0   100%
src/application/services.py        301     13    96%
src/application/views.py            29      0   100%
src/domain/__init__.py               0      0   100%
src/domain/agenda.py                97      0   100%
```

```
src/domain/common.py                    90      0   100%
src/domain/consulta.py                 160      0   100%
src/domain/events.py                   105      0   100%
src/domain/hospitalizacion.py          139      0   100%
src/domain/orden_servicio.py            76      0   100%
src/domain/paciente.py                 113      0   100%
src/domain/profilaxis.py                60      0   100%
src/domain/servicios.py                 40      0   100%
src/domain/specifications.py            72      0   100%
src/domain/tutor.py                     26      0   100%
src/infrastructure/__init__.py           0      0   100%
src/infrastructure/codec.py             54      2    96%
src/infrastructure/config.py            14      0   100%
src/infrastructure/db.py                25      1    96%
src/infrastructure/event_bus.py         20      0   100%
src/infrastructure/models.py            66      0   100%
src/infrastructure/notifier.py           9      0   100%
src/infrastructure/repositories.py     119      1    99%
src/infrastructure/seed.py              44      0   100%
src/infrastructure/uow.py               38      0   100%
TOTAL                                 2216     23    99%
```

Reproducible con `docker compose run --rm tests` o `pytest --cov=src --cov-fail-under=80`; el pipeline de GitHub Actions ejecuta el mismo comando con ese umbral.

Ejecución real de las reglas clave contra la API

```
### RN-04  POST /consultas/{id}/recetas  (penicilina, paciente alérgico)
HTTP 409
{"codigo":"AlergiaBloqueoError","mensaje":"BLOQUEO DE SEGURIDAD (RN-04): el paciente es alérgico a 'penicilina'"}

### RN-04 excepción: con justificación bajo cédula
HTTP 201

### RN-01  PUT /consultas/{id}/nota  (nota ya firmada)
HTTP 409
{"codigo":"NotaInmutableError","mensaje":"RN-01: la nota/receta está firmada y congelada; solo se admite una Nota
de Enmienda"}

### RN-01  POST /consultas/{id}/enmiendas
HTTP 201

### RN-06  alta médica (paso 1)
PENDIENTE_VALIDACION_CRUZADA | apto_para_egreso = False
### RN-06  liquidación (paso 2)
EGRESO_CONCLUIDO | apto_para_egreso = True
### jaula JP-01 libre:
True

### Paquete de datos  GET /ordenes/VET-0001
{'id': 'VET-0001', 'estado': 'SOLICITADA', 'total': '7587.00', 'puede_procesarse': True,
'elegible_beneficio_referencia': False, 'acciones_permitidas': ['aprobar', 'cancelar']}
### transición inválida: completar desde SOLICITADA
HTTP 409
{"codigo":"InvalidStateTransition","mensaje":"OrdenServicio: la acción 'completar' no está permitida en el estado
SOLICITADA"}

### US02  cirugía mayor
{'tipo': 'CIRUGIA', 'sala': 'QUIR-01', 'duracion_min': 90, 'estado': 'PROGRAMADA'}
### RNF-03  recepción pide expediente clínico: HTTP 403
```

12. Diferencias, supuestos y limitaciones

- **Autenticación:** `X-Rol` es solo demostrativo; en producción se sustituye por JWT/OIDC en un único punto (`security.requiere`).
- **Notificaciones** (WhatsApp/correo) simuladas; el mensaje de recordatorio no incluye enlace para confirmar o reagendar (requiere cliente web).
- **Laboratorio:** se registra metadato y huella SHA-256 del archivo, no el binario.
- **Rangos de signos vitales** (temperatura 37.5–39.5 °C, FC 60–180, FR 10–40) y el esquema de refuerzo por edad son supuestos técnicos **pendientes de validación clínica** con el Director Médico.
- **US13:** el Gherkin menciona «consulta cerrada hace más de 24 h»; el BRD/ADR no fija plazo, por lo que se permite la enmienda en cualquier momento posterior a la firma.
- **Fuera de alcance:** consentimiento informado, lista de espera de cancelaciones, pago en línea de estudios externos, inventario de biológicos, interfaz gráfica y alertas sonoras.
- **KPIs de negocio** (duplicados < 1 %, +40 % de retorno, código rojo < 15 s) son métricas de operación en producción; el sistema provee los datos pero no pueden demostrarse con datos ficticios.

13. Conclusiones

El Domain Model resultó adecuado: las reglas más delicadas (alergias, inmutabilidad, alta cruzada) quedaron concentradas en pocos métodos de dominio, probados al 100 % sin infraestructura, y la capa de aplicación se mantuvo como orquestadora. Specification y State Machine hicieron que las reglas del paquete de datos fueran explicables (cada rechazo nombra la regla incumplida) y verificables con un oráculo independiente. Probar sobre PostgreSQL real, y no solo SQLite, evitó un fallo de despliegue. La trazabilidad verificada por pruebas reduce el riesgo de documentación desalineada con el código.

Trabajo futuro: autenticación real; adaptador de mensajería; almacenamiento de archivos; consentimiento informado y lista de espera; validación clínica de rangos; cliente web/tablet responsivo (RNF-04); observabilidad.

Referencias

FIME UANL. (2026). *Caso 04: VetCare 24* (Unidad 2: Análisis de Requerimientos de Negocio). · Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley. · Evans, E. (2003). *Domain-Driven Design*. Addison-Wesley. · Documentos del equipo (U2/U3) en `docs/analisis/originales/`.

Anexo A. Cumplimiento de los requisitos del PIA

Criterio del documento del profesor	Evidencia en el repositorio	Sección
BRD completo y refinado de U2/U3	`docs/BRD.md` + originales en `docs/analisis/originales/`	2
≥ 15 User Stories con Gherkin, BPMN y glosario, con trazabilidad	15 historias; BPMN; 20 términos; `docs/TRAZABILIDAD.md`	3, 4, 5, 6
Modelo/diagramas DDD (entidades, VO, agregados, servicios, eventos)	Diagramas de clases por agregado; 23 eventos de dominio; servicios de dominio	7.2, 7.4
UML consistente con la solución	`docs/architecture/diagramas/*.png|.mmd` (derivados del código)	7.4
Patrones justificados	Tabla problema → patrón → alternativa descartada	7.3
SOLID demostrable	`docs/architecture/SOLID.md` + prueba de regla de dependencia (AST)	7.5
ADRs: contexto, decisión, alternativas, consecuencias	`docs/architecture/adr/ADR-001…006`	7.6
Estructura del repositorio (docs/, src/ en 4 capas, tests/, .github/workflows, docker-compose.yml, .env.example, README)	Presentes tal como lo indica el documento	8
Código funcional: DDD + Specification + State Machine + API REST	`src/domain`, `src/application`, `src/api`	7, 8
Persistencia/DB, OpenAPI/Swagger, casos de uso integrales	SQLite/PostgreSQL; 38 rutas; recorrido en `/docs`	8, 9
Pruebas unitarias e integración con cobertura ≥ 80 %	343 pruebas; 99.0 %	11
CI/CD activo en `.github/workflows/`	`ci.yml`	9
Docker Compose reproducible (DB + App + Tests)	`docker-compose.yml`, `Dockerfile`	9
Variables de entorno + despliegue + Swagger en producción	`.env.example`, `render.yaml`, `/docs`	9
Demo en vivo y defensa técnica	`docs/GUIA_DEFENSA.md` y presentación de apoyo	—
Historial de commits del equipo	Repositorio GitHub (commits de los tres integrantes)	Portada